Nature-Inspired Optimization Algorithms: Some Insights and Open Problems

Xin-She Yang

Middlesex University London

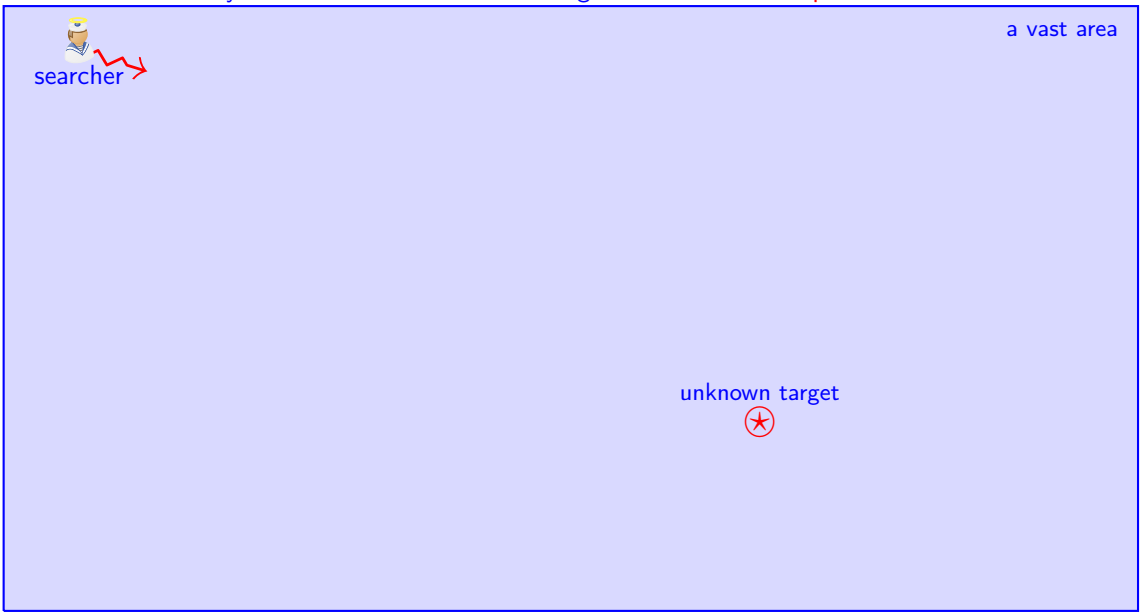
Google Scholar

x.yang (at) mdx.ac.uk

Research Gate

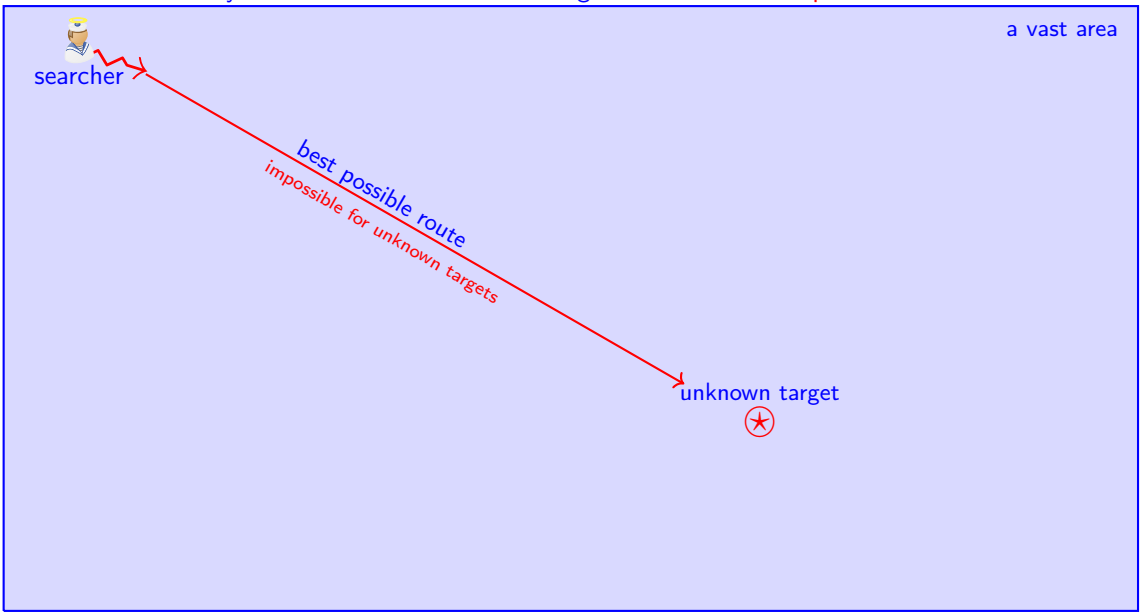
What is the best strategy to search for an unknown target in a vast region?

What is the best way to search for the unknown target in a vast search space?



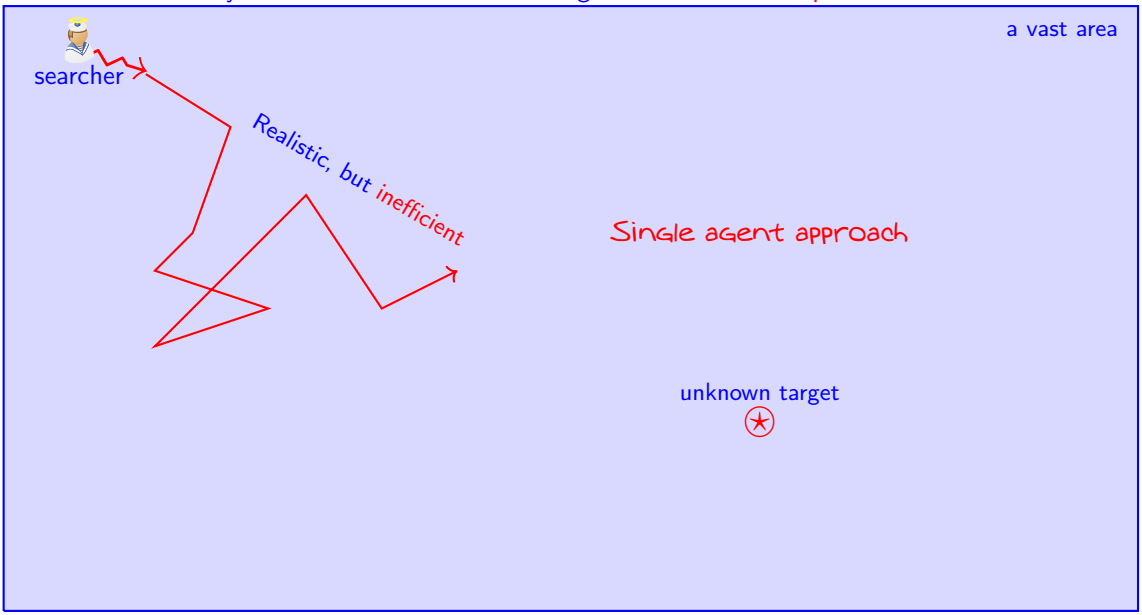
e.g., locating a missing object in a vast ocean, searching for a potential fire in a large forest, etc.

What is the best way to search for the unknown target in a vast search space?



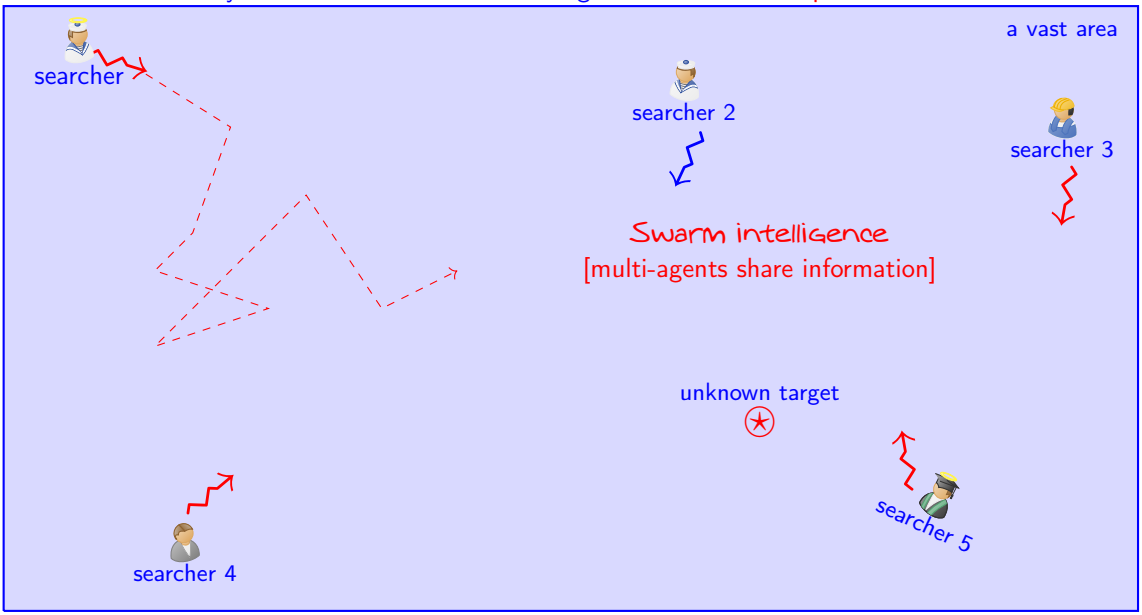
e.g., locating a missing object in a vast ocean, searching for a potential fire in a large forest, etc.

What is the best way to search for the unknown target in a vast search space?



e.g., locating a missing object in a vast ocean, searching for a potential fire in a large forest, etc.

What is the best way to search for the unknown target in a vast search space?



e.g., locating a missing object in a vast ocean, searching for a potential fire in a large forest, etc.

What is the best way to search for the unknown target in a vast search space?

e.g., locating a missing object in a vast ocean, searching for a potential fire in a large forest, etc.

Almost Everything is Optimization

Almost everything is optimization ... or needs optimization ...

- Maximize efficiency, accuracy, profit, performance, sustainability, ...
- Minimize costs, wastage, energy consumption, travel distance/time, CO_2 emission, impact on environment, ...

Mathematical Optimization

Objective $f(\boldsymbol{x}) = [f_1(\boldsymbol{x}), f_2(\boldsymbol{x}), ..., f_p(\boldsymbol{x})]$, $\boldsymbol{x} \in \mathbb{R}^D$

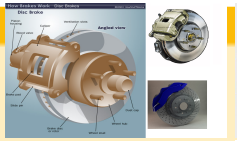
subject to multiple equality and/or inequality constraints:

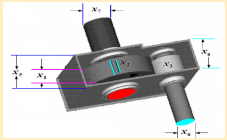
$$h_i(\boldsymbol{x}) = 0, \quad (i = 1, 2, ..., M),$$

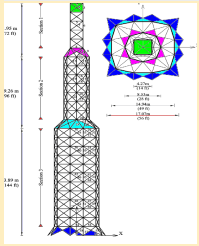
$$g_j(\boldsymbol{x}) \leq 0, \quad (j = 1, 2, ..., N).$$

Single-objective optimization problems (if $p = 1$).

In general, objectives and constraints can be highly nonlinear.

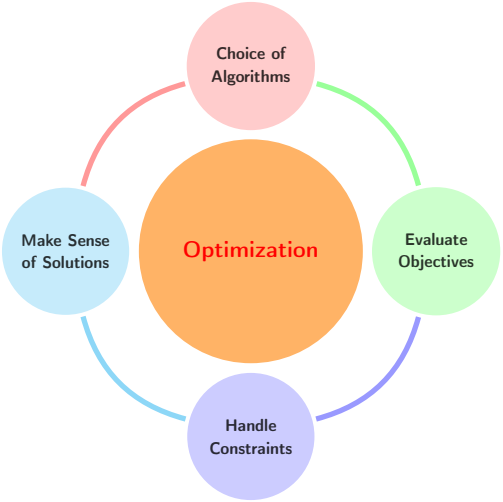






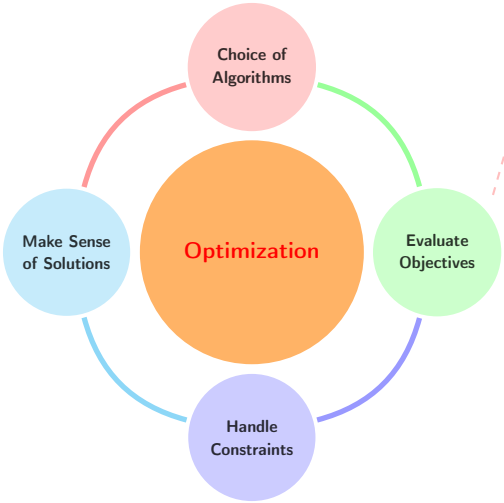
Optimization: Key Components and Challenges

To find the optimal solutions of an optimization problem is like 'treasure-hunting' in a vast area. Though we may have a metal detector to explore, we have no idea where the treasure/gold is.



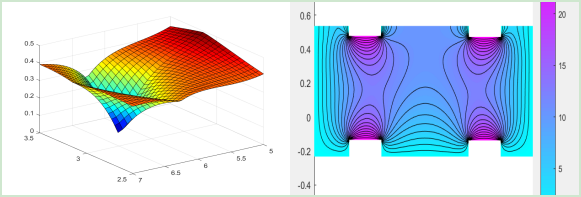
Optimization: Key Components and Challenges

To find the optimal solutions of an optimization problem is like 'treasure-hunting' in a vast area. Though we may have a metal detector to explore, we have no idea where the treasure/gold is.



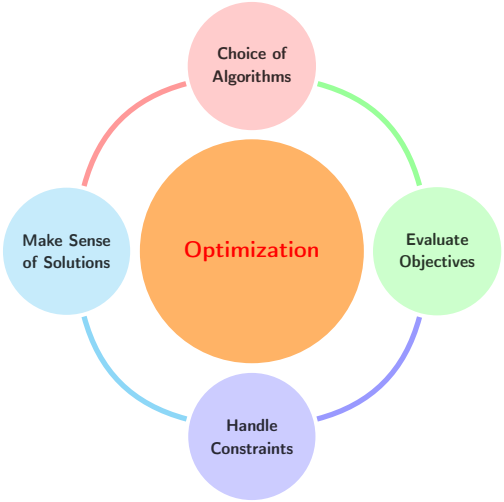
Evaluations of Objectives by Simulation

- Objective evaluations by simulation are the most computationally extensive part
- Use an external software tool/simulator (e.g., FEM, FVM, etc.)
- Often call the simulator many times



Optimization: Key Components and Challenges

To find the optimal solutions of an optimization problem is like 'treasure-hunting' in a vast area. Though we may have a metal detector to explore, we have no idea where the treasure/gold is.



Once an optimization problem is formulated correctly, we have to solve it with the following steps/challenges:

- What algorithm(s) should we use?
- How to handle both equality and inequality constraints effectively?
- How to evaluate the objectives efficiently?
- How to interpret/make sense of the solutions?

Here, we focus on the algorithms.

Why gradient-based algorithms and Monte Carlo are not enough?

Gradient-based methods, including moment-based/momentum-based optimizers

- Require multiple computations of derivatives, which can be time-consuming.
- The final solutions can be dependent on the initial guess/starting point.
- They are usually efficient, but no guarantee of global optimality.

Monte Carlo

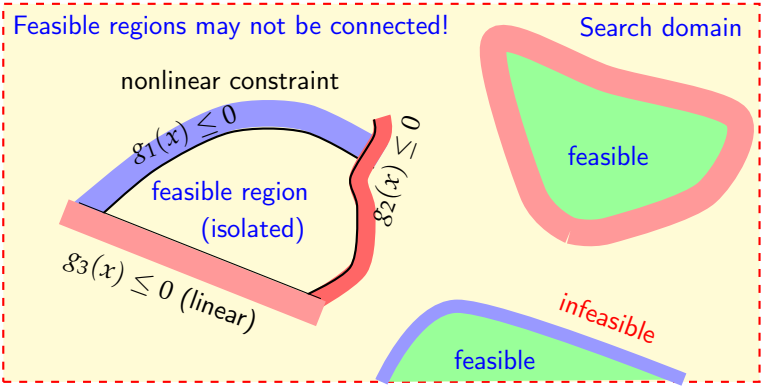
- Convergence rates are typically $O(1/\sqrt{N})$. Quasi-Monte Carlo $O(1/N)$.
- The target search regions have a nearly zero volume ratio, especially in high-dimensional spaces.
- Pseudo-random, even quasi-random, distribution of solution vectors are not efficient.

Need to go a bit further, also with a bit of swarm intelligence

- Try to use multi-agents to carry out parameter search.
- Require some selection mechanisms to generate better solution sets.
- Need to use landscape information, memory and collective intelligence (if any).

$\Longrightarrow$ Nature-inspired optimization algorithms/metaheuristics.

Optimization problems can be highly nonlinear, multimodal, subject to multiple constraints with irregular parameter spaces.



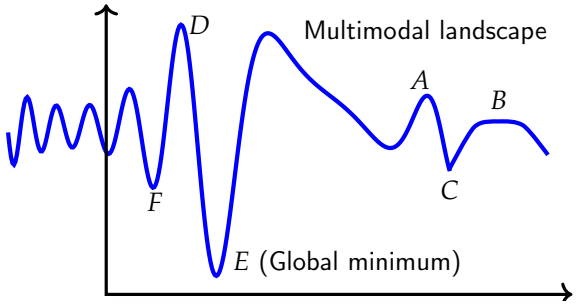
Only 3 types of problems that can be solved efficiently.

We can solve only 3 types of optimization problems (Boyd and Vandenberghe, 2004):

- Linear programming
- Convex optimization
- Problems that can be converted into the other two types (e.g., method of least squares, sequential quadratic programming with approximations)

Everything else seems difficult, especially large-scale, nonlinear, multimodal problems.

Optimality

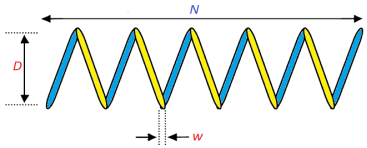


Strong and Weak Optimality

- Point A is a strong local maximum, whereas point B within a small flat region (potentially many x_*) is a weak local maximum.
- Point D is the global maximum, and point E is the global minimum.
- Point F is a strong local minimum.

This optimality implicitly assumes that condition that all constraints are satisfied. Otherwise, the optimality is not valid.

Spring Design (Example 1)



The wire diameter w (or x_1), mean coil diameter D (or x_2) and the number N (or x_3) of active coils.

$$\min f(\boldsymbol{x}) = (2 + x_3) x_1^2 x_2, \tag{1}$$

subject to

$$g_1(\boldsymbol{x}) = 1 - \frac{x_2^3 x_3}{71785 x_1^4} \leq 0, \tag{2}$$

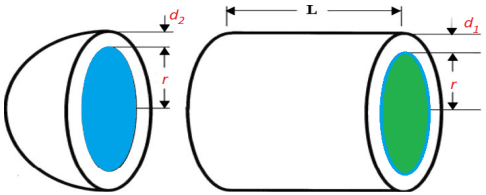
$$g_2(\boldsymbol{x}) = \frac{4x_2^2 - x_1 x_2}{12566(x_2 x_1^3 - x_1^4)} + \frac{1}{5108 x_1^2} - 1 \leq 0, \tag{3}$$

$$g_3(\boldsymbol{x}) = 1 - \frac{140.45 x_1}{x_2^2 x_3} \leq 0, \tag{4}$$

$$g_4(\boldsymbol{x}) = \frac{x_1 + x_2}{1.5} - 1 \leq 0. \tag{5}$$

It's an open problem – true global optimality is unknown.

Pressure Vessel Design (Example 2)



The objective is to minimize the total cost of a pressure vessel:

$$\text{minimize } f(\boldsymbol{x}) = 0.6224rLd_1 + 1.7781r^2d_2 + +3.1661Ld_1^2 + 19.84rd_1^2, \tag{6}$$

subject to four constraints:

$$g_1(\boldsymbol{x}) = -d_1 + 0.0193r \le 0, \quad g_2(\boldsymbol{x}) = -d_2 + 0.00954r \le 0, \tag{7}$$

$$g_3(\boldsymbol{x}) = -\frac{4\pi r^3}{3} - \pi r^2 L - 1296000 \le 0, \quad g_4(\boldsymbol{x}) = L - 240 \le 0. \tag{8}$$

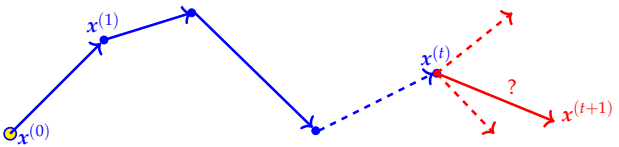
The simple limits for the inner radius and length are: $10.0 \le r, L \le 200.0$, while the thickness can only take values of the integer multiples of 0.0625 inch

$$1 \times 0.0625 \le d_1, d_2 \le 99 \times 0.0625. \tag{9}$$

Global optimality is known [Yang et al., Int. J. Bio-Inspired Computation, 5 (6), 329–335 (2013)].

Essence of an Optimization Algorithm

To generate a better solution point $\boldsymbol{x}^{(t+1)}$ (a solution vector) from an existing solution $\boldsymbol{x}^{(t)}$. That is, $\boldsymbol{x}^{(t+1)} = A(\boldsymbol{x}^{(t)}, \alpha)$. [Ideally, in a few steps]



For example, Newton-Raphson

$$\boldsymbol{x}^{(t+1)} = \boldsymbol{x}^{(t)} - \frac{\nabla f(\boldsymbol{x}^{(t)})}{\nabla^2 f(\boldsymbol{x}^{(t)})}$$

Newton's Method for optimization

$$x_{t+1} = x_t - \frac{f'(x_t)}{\nabla^2 f(x_t)} = x_t - \boldsymbol{H}^{-1} \nabla f,$$

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}, \quad \boldsymbol{H} = \nabla^2 f = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix},$$

where ∇f is the gradient vector and $\boldsymbol{H}$ is the Hessian matrix. Newton's method is also called Newton-Raphson method.

Different algorithms $\Longrightarrow$ Different ways for generating new solutions!

What's Wrong with Traditional Algorithms?

- Though traditional algorithms are efficient, they are mostly local search, thus cannot guarantee global optimality (except for linear and convex optimization).
- Results often depend on the initial starting points (except for linear and convex problems).

Nature-Inspired Optimization Algorithms

Heuristic or metaheuristic algorithms tend to be global optimizers so as to increase the probability of finding the global optimality (as a global search tool), though they can be potentially more computationally expensive.

Nature-Inspired Algorithms/Evolutionary Computation

- Genetic algorithms (1960s/1970s), evolutionary strategy (Rechenberg & Swefel 1960s), evolutionary programming (Fogel et al. 1960s).
- Simulated annealing (Kirkpatrick et al. 1983), Tabu search (Glover 1980s), ant colony optimization (Dorigo 1992), genetic programming (Koza 1992), particle swarm optimization (Kennedy & Eberhart 1995), differential evolution (Storn & Price 1996/1997), harmony search (Geem et al. 2001), artificial bee colony (Karaboga, 2005).
- Firefly algorithm (Yang 2008), cuckoo search (Yang & Deb 2009), bat algorithm (Yang, 2010), flower pollination algorithm (2012).

It is estimated that there are 540 different metaheuristic algorithms in the current literature[1]. Even more algorithms are emerging every year ...

For details, please refer to:

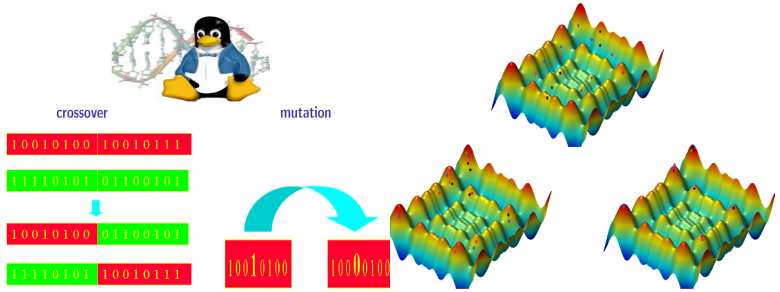
- Xin-She Yang, *Nature-Inspired Optimization Algorithms*, 2nd Edition, Academic Press, (2020).
- Xin-She Yang, Optimization Techniques and Applications with Examples, Wiley, (2018).
- Xin-She Yang, Introduction to Algorithms for Data Mining and Machine Learning, Academic Press/Elsevier, (2019). [Slides are available for teaching a course]

[1] Rajwat et al. (2023). An exhaustive review of the metaheuristic algorithms for search and optimization, Artificial Intelligence Review, vol. 56, 13187–13257.

Nature-Inspired Algorithms

Genetic Algorithm (GA)

Starting with random sampling, the population will gradually converge.



Extensive research on GA (a well-studied algorithm)

A wide range of applications and some rigorous theoretical analysis
[e.g., mutation rate by Belavkin (2012), convergence by Greenhalgh and Marshal (2000)]

- R Belavkin, Dynamics of information and optimal control of mutation in evolutionary systems. *Dynamics of Information Systems: Mathematical Foundations* (edited by A Sorokin et al.), (2012).
- D Greenhalgh and S Marshal, Convergence criteria for genetic algorithms. SIAM J Comput, 30(1), 269-282 (2000).

PSO

Particle Swarm Optimization (Kennedy and Eberhart, 1995)

$$\begin{cases} v_i^{t+1} = v_i^t + \alpha\epsilon_1(g^* - x_i^t) + \beta\epsilon_2(x_i^* - x_i^t), & (10) \\ x_i^{t+1} = x_i^t + v_i^{t+1}. & (11) \end{cases}$$

α, β = learning parameters, ϵ_1, ϵ_2=random numbers.
v_i = velocity, x_i = position of particle i (a solution vector).

Quite efficient, but it can have premature convergence.

More than 20 variants and various degrees of improvements.

Insight 1: A Linear Dynamical System

PSO is a linear system with random perturbations (Clerc and Kennedy, 2002).

Firefly Algorithm —**Insight 2: A Nonlinear Dynamical System**

Firefly Behaviour and Idealization (Yang, 2008)

- Fireflies are unisex and brightness varies with distance.
- Less bright ones will be attracted to brighter ones.
- If no brighter fireflies can be seen in a neighborhood, a firefly will move randomly in the space.

$$x_i^{t+1} = x_i^t + \beta_0 e^{-\gamma r_{ij}^2}(x_j - x_i) + \alpha\, \epsilon_i^t. \tag{12}$$

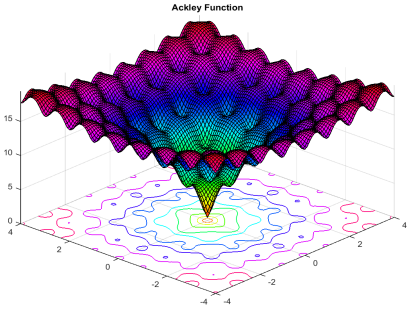
◇ The objective landscape maps to a light-based landscape, and fireflies swarm into the brightest points/regions.

◇ There is no g_*, therefore, there is no leader in the population. Also, a highly nonlinear iterative system, so subdivision into multiswarms is possible.

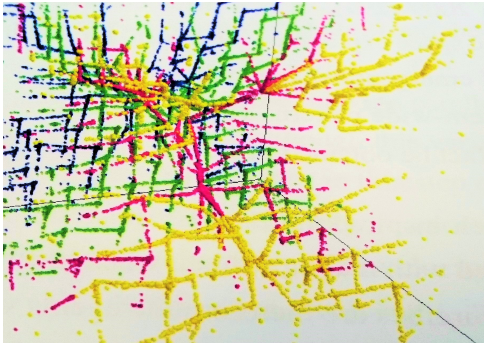
Why is FA so efficient? — **Insight 3: Fractal-like search paths**

FA is a nonlinear system with richer dynamical features

- Automatically subdivide the whole population into subgroups (subswarms), and each subgroup swarms around a local mode/optimum.
- Suitable for multimodal, nonlinear, global optimization problems.



Ackley's Function ($\gamma = 2$)



Tuned parameters ($\gamma = 0.5$)

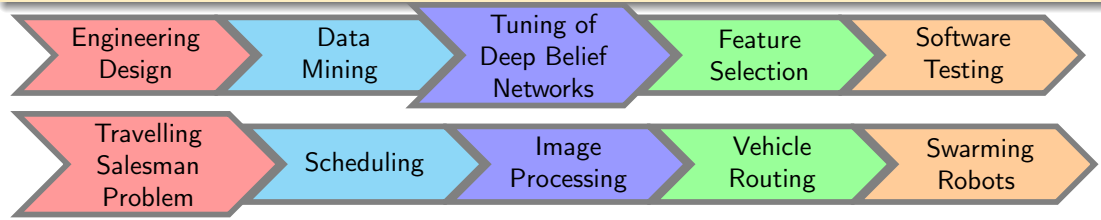
Other Algorithms

There are more than 540 algorithms and variants in the current literature, including ant colony optimization, cuckoo search, differential evolution, memetic algorithm, ... (Expanding literature...)

Reviews

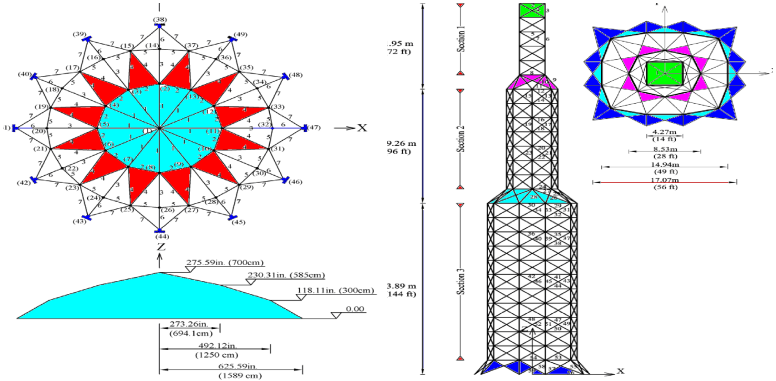
- Xin-She Yang, *Nature-Inspired Optimization Algorithms*, 2nd Edition, Academic Press/Elsevier, (2020).
- Xin-She Yang, *Nature-Inspired Computation and Swarm Intelligence: Algorithms, Theory and Applications*, Academic Press/Elsevier, (2020).
- Xin-She Yang, Nature-inspired optimization algorithms: Challenges and open Problems, *Journal of Computational Science*, Article 101104, (2020).

Do they really work? **They can indeed work surprisingly well in practice.**



Dome and Tower Designs (Example 1)

Large-scale real-world structural designs (e.g., dome, tower).
Objective: to minimize total steel weight/costs.

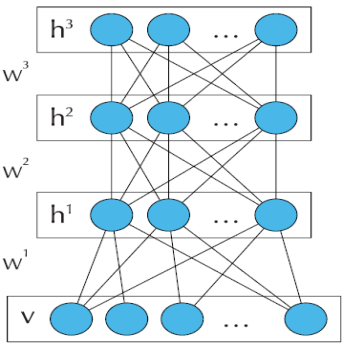


120-bar dome: Divided into 7 groups, 120 design elements, about 200 constraints.
26-storey tower: 942 design elements, 244 nodal links, > 4000 nonlinear constraints.

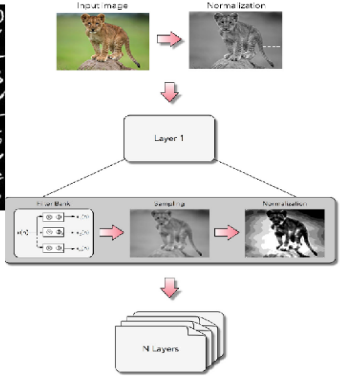
X.-S. Yang and A. H. Gandomi, Bat algorithm: a novel approach for global engineering optimization, Engineering Computations, 29(5), 464-483 (2012).

Deep Belief Networks Fine Tuning (Example 2)

Tuning hyper-parameters in deep belief networks by the bat algorithm (BA) achieved the best accuracy.



The DBN architecture used in this work.



Papa, Rosa, Pereira, Yang, Quaternion-based deep belief networks fine tuning, Applied Soft Computing, 60, 328–335 (2017).

Optimal Transmit Beamforming for Wireless Communications (Example 3)

For a given cognitive wireless communication system, consisting of a base station with M attentae, K single-antenna prime users, and U active single-antenna secondary users, the objective is to design the downlink beamforming vector so as to minimize the cognitive base station transmit power

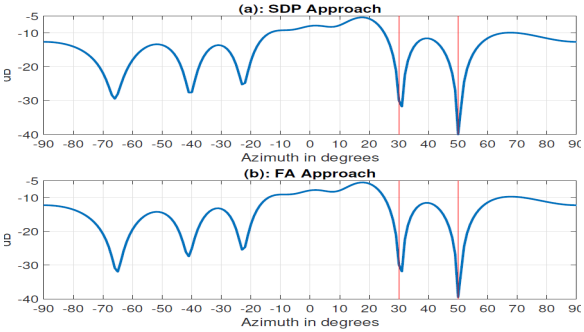
$$\min \sum_{t=1}^{U} \mathbf{w}_t^H,$$

subject to

$$\frac{\mathbf{w}_t^H \boldsymbol{R} \mathbf{w}_t}{\sum_{j=1, j\neq t}^{U} \mathbf{w}_j^H \boldsymbol{R} \mathbf{w}_j + \sigma_t^2} \geq \eta_t,$$

$$\sum_{j=1}^{U} \mathbf{w}_j^H \boldsymbol{R} \mathbf{w}_j \leq I_k,$$

where η_t is the required SINR level for user t, and σ^2 is the variance of the noise.



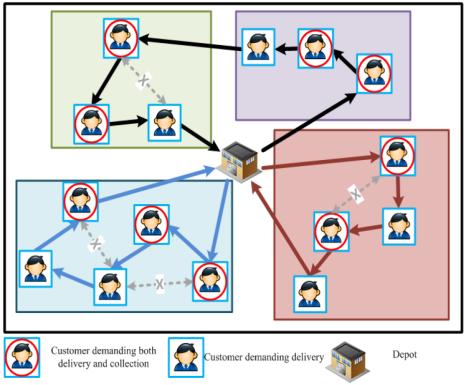
The Firefly Algorithm has less computational complexity, and has obtained equally good results as the traditional semidefinite programming (SDP).

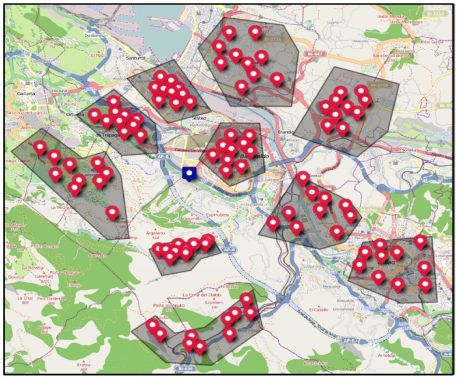
T. Le and X.-S. Yang, Generalized firefly algorithm for optimal transmit beamforming, *IEEE Transactions on Wireless Communications*, (2023). http://doi.org/10.1109/TWC.2023.3328713

Vehicle Routing with Real Traffic Information (Example 4)

Optimal deployment of vehicles to distribute goods (e.g., medical supplies, newspapers) and collect recyclable goods, subject to dynamic real-time traffic (e.g., regions of northern Spain).

The discrete firefly algorithm (DFA) obtained the best results.



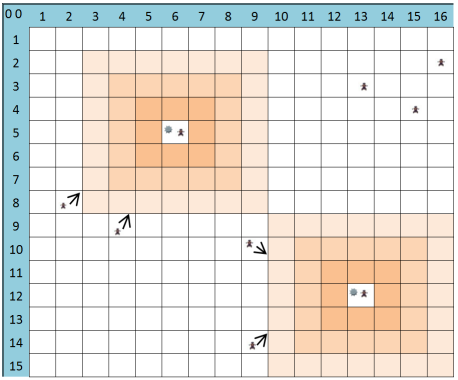


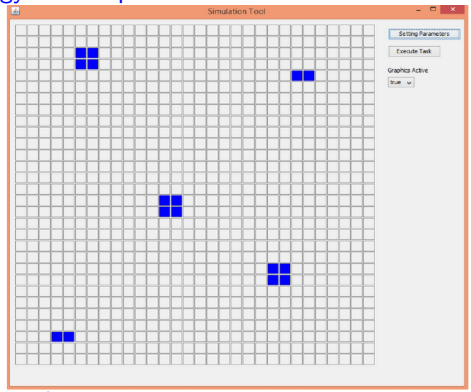
E. Osaba, X.S. Yang, F. Diaz, et al., A discrete firefly algorithm to solve a rich vehicle routing problem with recycling policy, Soft Computing, 21(18), 5295-5308 (2017).

Swarm Robots (Example 5)

A swarm of mobile robots, with wireless distributed protocol and no central decision, explore the unknown targets in a vast region so as to find all the (unknown) targets and handle/remove these targets safely and quickly.

The objectives are to minimize search time and energy consumption.

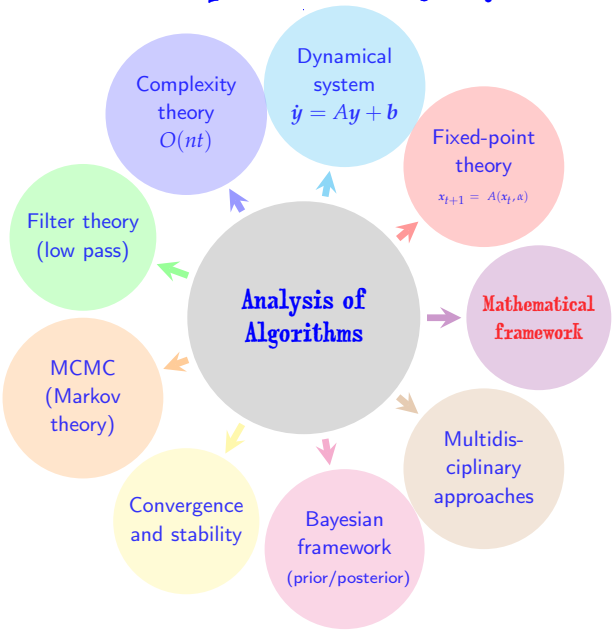




A hybrid approach using the firefly algorithm and ant colony optimization.

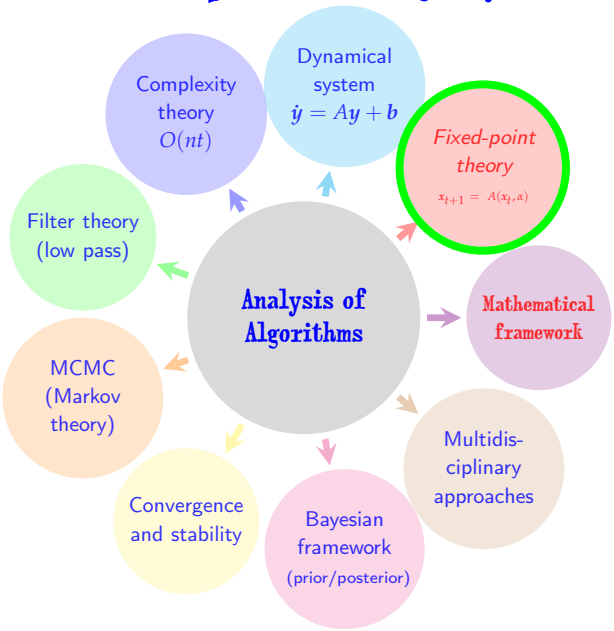
F. De Rango, N. Palmieri, X.S. Yang, S. Marano, Swarm robotics in wireless distribution protocol design for coordinating robots involved in cooperative tasks, Soft Computing, 22 (13), 4251-4266 (2018).

Insight 4: Algorithm structure and parameter settings may determine its performance.



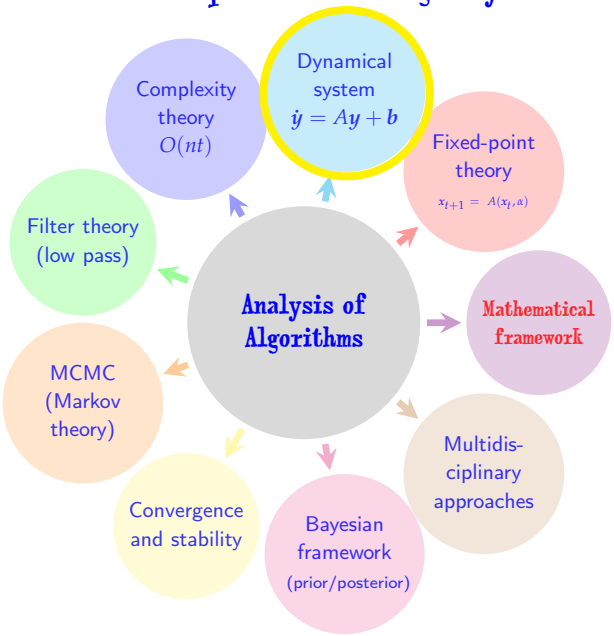
X. S. Yang and X. He, **Mathematical Foundations of Nature-Inspired Algorithms**, Springer, (2019).

Insight 4: Algorithm structure and parameter settings may determine its performance.



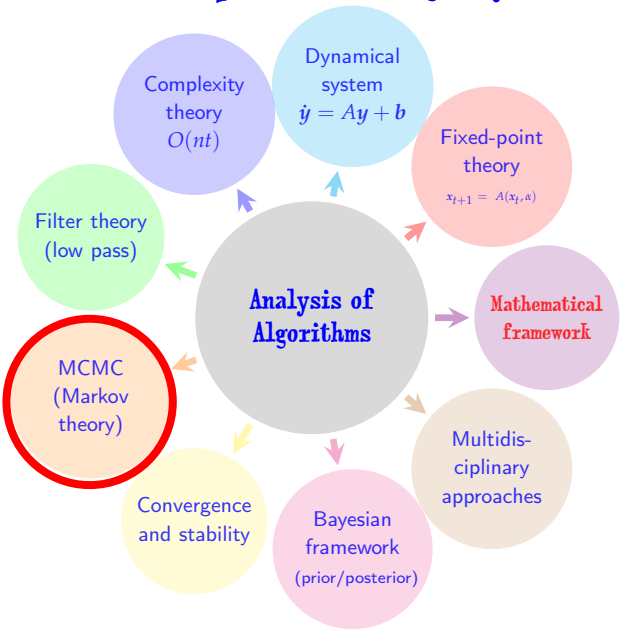
X. S. Yang and X. He, **Mathematical Foundations of Nature-Inspired Algorithms**, Springer, (2019).

Insight 4: Algorithm structure and parameter settings may determine its performance.



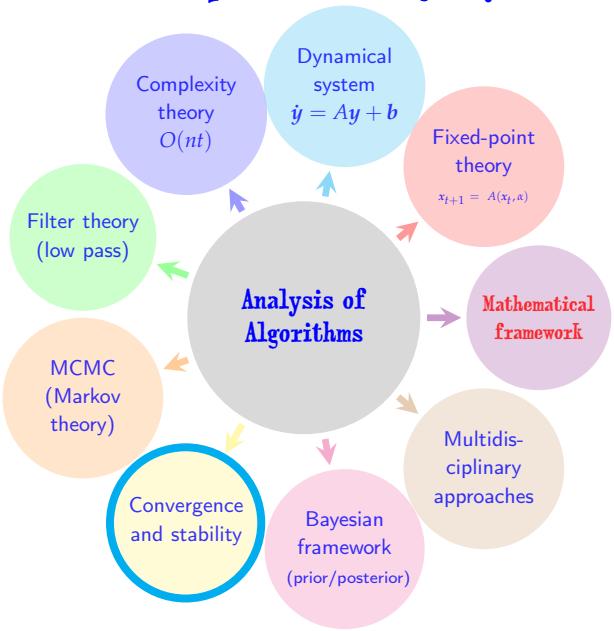
X. S. Yang and X. He, **Mathematical Foundations of Nature-Inspired Algorithms**, Springer, (2019).

Insight 4: Algorithm structure and parameter settings may determine its performance.



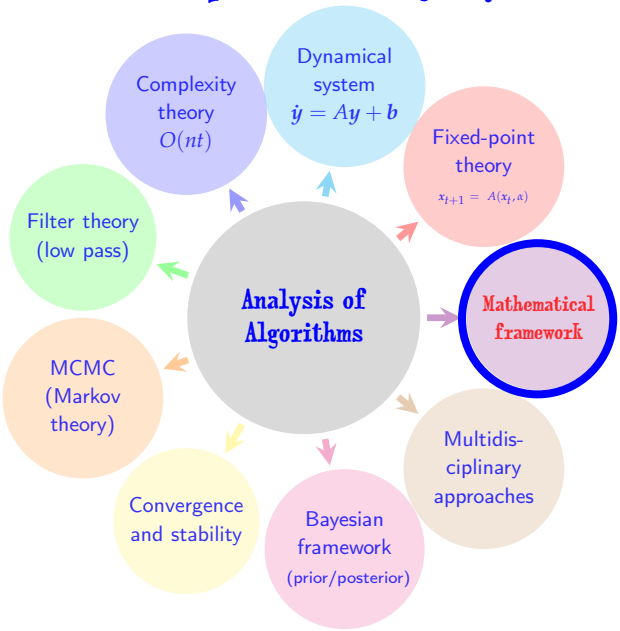
X. S. Yang and X. He, **Mathematical Foundations of Nature-Inspired Algorithms**, Springer, (2019).

Insight 4: Algorithm structure and parameter settings may determine its performance.



X. S. Yang and X. He, **Mathematical Foundations of Nature-Inspired Algorithms**, Springer, (2019).

Insight 4: Algorithm structure and parameter settings may determine its performance.



X. S. Yang and X. He, **Mathematical Foundations of Nature-Inspired Algorithms**, Springer, (2019).

Theoretical Analysis (Example 1: Bat Algorithm)

Global convergence analysis of the Bat Algorithm as a dynamical system.

The main equations of the bat algorithm can be simplified as

$$\begin{cases} f_i = f_{\min} + (f_{\max} - f_{\min})\beta, \\ \boldsymbol{v}_i^{(k+1)} = \boldsymbol{v}_i^{(k)} + (g - \boldsymbol{x}_i^{(k)})f_i, \\ \boldsymbol{x}_i^{(k+1)} = \boldsymbol{x}_i^{(k)} + \boldsymbol{v}_i^{(k+1)}. \end{cases}$$

We rewrite them more generally as

$$\boldsymbol{v}_i^{(k+1)} = \theta \boldsymbol{v}_i^{(k)} + \mu(g - \boldsymbol{x}_i^{(k)}), \quad \boldsymbol{x}_i^{(k+1)} = \boldsymbol{x}_i^{(k)} + \boldsymbol{v}_i^{(k+1)},$$

where θ and μ are parameters. This system can equivalently be written as

$$Y_{k+1} = AY_k + Mg, \quad Y_k = \begin{bmatrix} \boldsymbol{x}_i^{(k)} \\ \boldsymbol{v}_i^{(k)} \end{bmatrix}, \quad A = \begin{bmatrix} 1-\mu & \theta \\ -\mu & \theta \end{bmatrix}, \quad M = \begin{bmatrix} \mu \\ \mu \end{bmatrix}.$$

Lyapunov stability requires that $|\lambda| < 1$ and

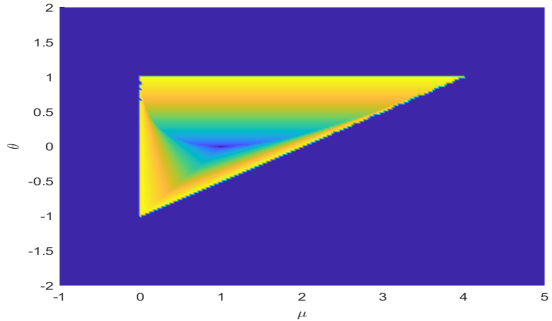
$$\det \begin{bmatrix} 1-\mu-\lambda & \theta \\ -\mu & \theta-\lambda \end{bmatrix} = 0.$$

Insight 5: Tuning parameter and parameter settings are important.

The parameter ranges (for the Bat Algorithm) become

$$\begin{cases} -1 \le \theta \le +1, \\ \mu \ge 0, \\ 2\theta - \mu + 2 \ge 0, \end{cases}$$

which is a triangular region for stability.



S. Chen, G. Peng, X. He, X.S. Yang, Global convergence analysis of the bat algorithm using a Markovian framework and dynamical system theory, *Expert Systems and Applications*, 114, 173-182 (2018).

Theoretical Analysis (Example 2: Cuckoo Search)

Markov Chains and Convergence

- The next state or solutions only depends on the current state (or current solutions) and transition probability (ways of modifications).
- The final distribution of the converged chain will 'forget' its initial states.
- Convergence is in the probability sense (with probability one).

Global Convergence of Cuckoo Search Algorithm

$$\begin{cases} \boldsymbol{x}_i^{t+1} \leftarrow \boldsymbol{x}_i^t, & \text{if } r < p_a, \\ \boldsymbol{x}_i^{t+1} \leftarrow \boldsymbol{x}_i^t + \alpha \otimes L(\lambda), & \text{if } r \geq p_a, \end{cases} \tag{13}$$

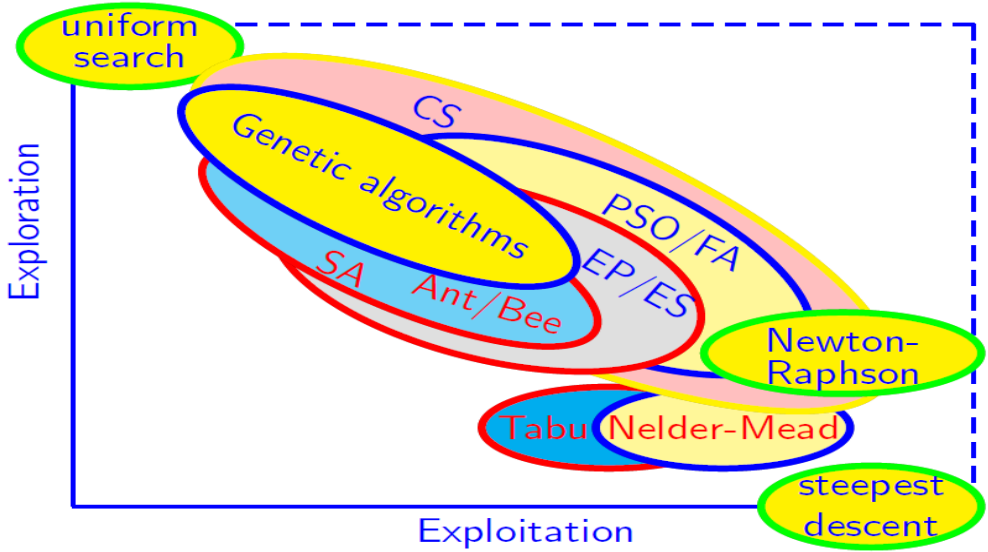
The convergence probability to the global optimal set G is

$$\lim_{t \to \infty} P(\boldsymbol{x}^t \in G) \to 1. \quad \text{(Almost surely!)} \tag{14}$$

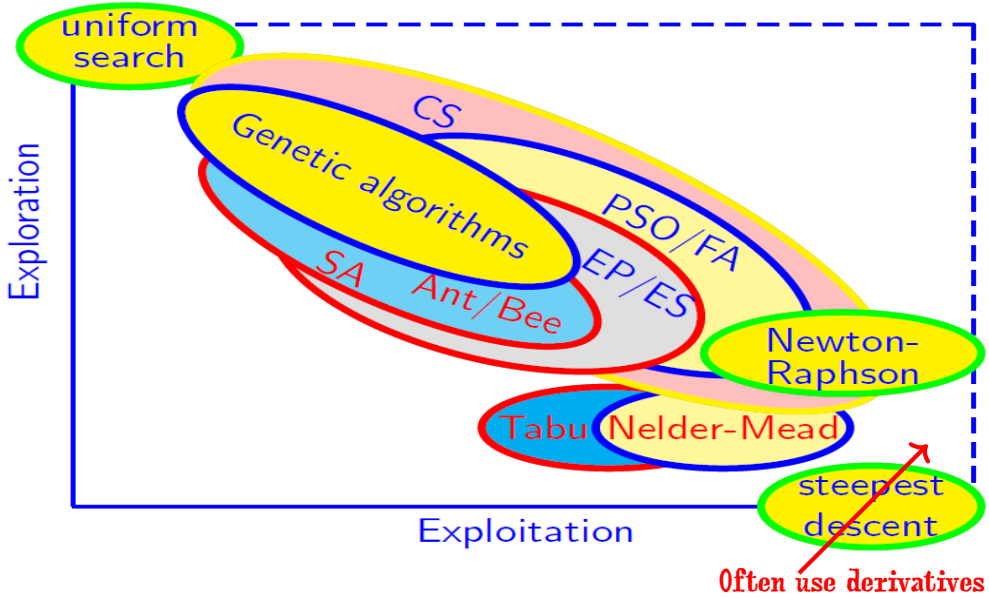
This theory still allows a small probability of no convergence for a finite number of iterations.

He, Wang, Yang, Global convergence analysis of cuckoo search using Markov theory, in: *Nature-Inspired Algorithms and Applied Optimization*, Springer Studies in Computational Intelligence SCI744, 53-67 (2018).

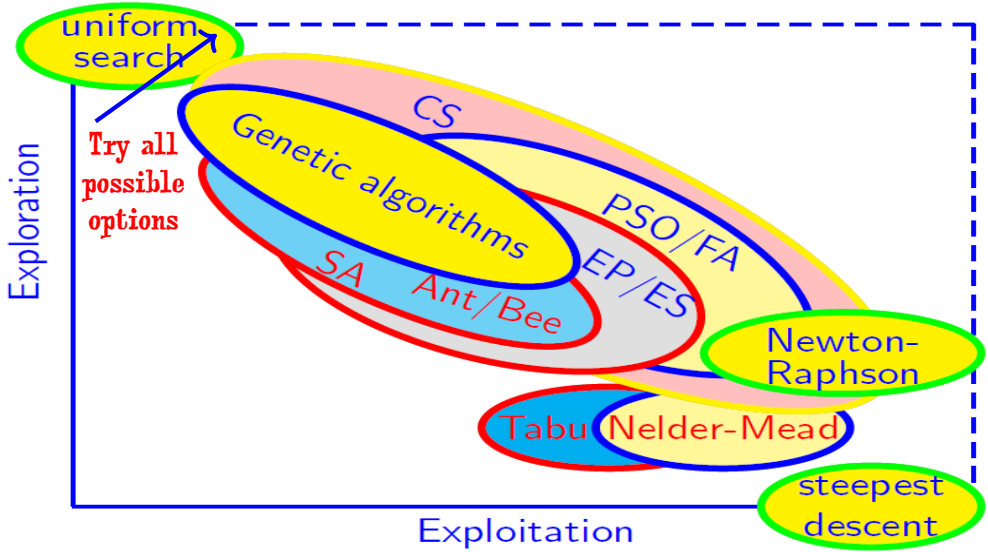
Insight 6: Balancing of Exploration and Exploitation is Important



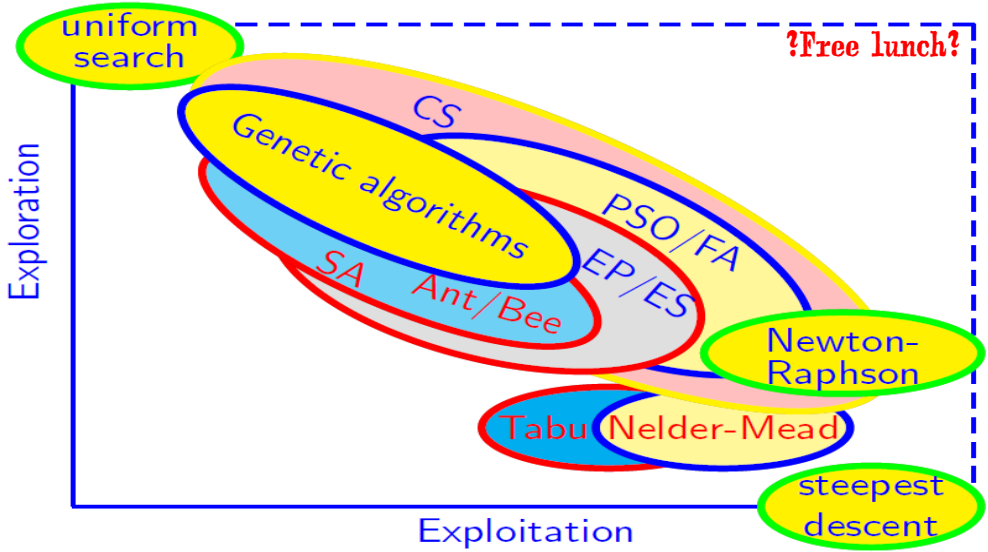
Insight 6: Balancing of Exploration and Exploitation is Important



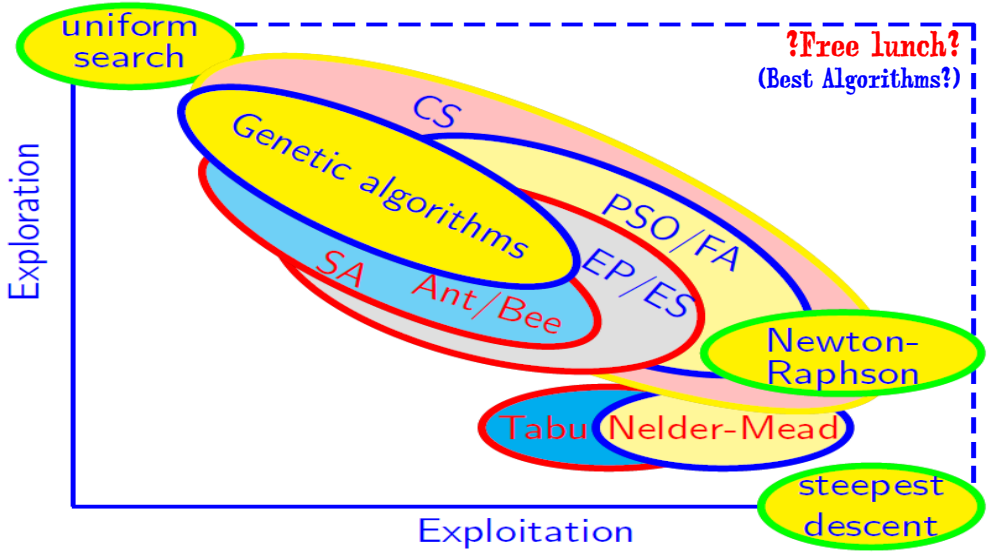
Insight 6: Balancing of Exploration and Exploitation is Important



Insight 6: Balancing of Exploration and Exploitation is Important



Insight 6: Balancing of Exploration and Exploitation is Important



Discussions: Open Problems

- Open Problem #1: How to build a unified theoretical framework for analyzing all nature-inspired algorithms mathematically, to gain insights about their convergence, rate of convergence, stability and robustness?

  Is it possible to build a set of algorithmic framework to unify all the exist algorithms (or at least most of them)?

Convergence does not guarantee true optimality

For example, the swarm population can converge (under certain conditions), such as the diversity or variance approaching to zero. However, the converged solutions are not necessarily the optimal solution(s) to the problem of interest.

Discussions: Open Problems

- Open Problem #1: How to build a unified theoretical framework for analyzing all nature-inspired algorithms mathematically, to gain insights about their convergence, rate of convergence, stability and robustness?

  Is it possible to build a set of algorithmic framework to unify all the exist algorithms (or at least most of them)?

Convergence does not guarantee true optimality

For example, the swarm population can converge (under certain conditions), such as the diversity or variance approaching to zero. However, the converged solutions are not necessarily the optimal solution(s) to the problem of interest.

- Open Problem #2: How to best tune the parameters of a given algorithm to achieve its best performance for a given set of problems?

  How to vary or control these parameters so as to maximize the algorithm's performance?

Tuning can be problem dependent

Even an algorithm is well tuned for one set of problems, there is no guarantee that such setting is the best for the algorithm to solve other problems.

Discussions: Open Problems

- Open Problem #3: What types of benchmarking are useful?
  Do free lunches exist? Under what conditions?

How useful are test functions?

Maybe, very limited. Test functions are usually unconstrained and smooth.

Real-world problems can be very different from such smooth functions.

Ranking does not mean free lunches! [In essence, ranking is zero sum.]

Ranking can heavily depend on the problems and algorithms used.

Discussions: Open Problems

- Open Problem #3: What types of benchmarking are useful?
  Do free lunches exist? Under what conditions?

How useful are test functions?

Maybe, very limited. Test functions are usually unconstrained and smooth.

Real-world problems can be very different from such smooth functions.

Ranking does not mean free lunches! [In essence, ranking is zero sum.]

Ranking can heavily depend on the problems and algorithms used.

- Open Problem #4: What are the most suitable performance metrics for fairly comparing all nature-inspired algorithms for optimization ?

  Is it possible to design a unified framework to compare all algorithms fairly and rigorously?

Performance depends on the performance metric(s)

One cannot compare a marathon athlete with a tennis player.

Discussions: Open Problems

- Open Problem #5: How to best scale up the algorithms that work well for small-scale problems to solve truly large-scale problems in real-world applications efficiently?

Parallelization and high-performance computing may not be enough.

Large-scale problems can be very time-consuming. Better algorithms are highly needed.

Discussions: Open Problems

- Open Problem #5: How to best scale up the algorithms that work well for small-scale problems to solve truly large-scale problems in real-world applications efficiently?

Parallelization and high-performance computing may not be enough.

Large-scale problems can be very time-consuming. Better algorithms are highly needed.

Rise of true swarm intelligence

What are the main conditions for swarm intelligence?

How to achieve these conditions?

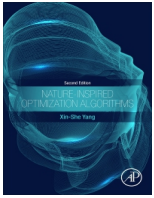
Can algorithms be truly intelligent (without learning or data)?

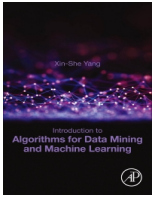
For details, please refer to:

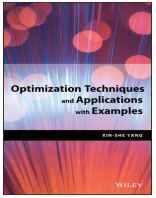
Xin-She Yang, Nature-inspired optimization algorithms: challenges and open problems, *Journal of Computational Science*, Article 101104, (2020).

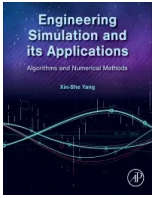
Thank you :) Any questions?

- Xin-She Yang, Nature-Inspired Optimization Algorithms, 2nd Edition, Academic Press/Elsevier, (2020). [Slides are available for teaching]
- Xin-She Yang, Engineering Simulation and its Applications: Algorithms and Numerical Methods, Academic Press/Elsevier, (2024).
- Xin-She Yang, Introduction to Algorithms for Data Mining and Machine Learning, Academic Press/Elsevier, (2019). [Slides are available for teaching a course]
- Xin-She Yang, Optimization Techniques and Applications with Examples, John Wiley & Sons, (2018).
- Xin-She Yang, Nature-inspired optimization algorithms: Challenges and open problems, *Journal of Computational Science*, Article 101104, (2020).









Google Scholar

x.yang @ mdx.ac.uk

Web